

下面是基于当前代码整理的项目功能和请求调用流程。当前项目同时存在“正式 OAuth/数据库功能”和一部分“旧版原型接口”，使用时要注意接口前缀不同。

一、项目整体功能

项目是一个基于知乎账号的数字分身社交平台，主要包含：

1. 知乎 OAuth 登录
2. 知乎资料读取
3. 数字分身初始化
4. 性格测评
5. 兴趣和专长领域选择
6. 数字分身画像和版本管理
7. 场景选择
8. 三种 Agent 匹配方式
9. 双 Agent 自动对话
10. 共同观点话题筛选
11. 接收方兴趣判断和低兴趣拒绝
12. 对话实时 SSE 推送
13. 对话取消和最多 10 轮限制
14. 对话契合度评估
15. 聊天组持久化
16. 聊天记录查询
17. 聊天记录未读状态
18. 场景 Agent 占用控制
19. 次日状态切换
20. 行为记忆写入
21. PostgreSQL 与 Chroma 混合检索
22. 前端通过 Nginx 或本地端口访问后端

二、前端和后端地址

当前本机直连模式配置为：

```
前端: http://127.0.0.1:3000
后端: http://127.0.0.1:8000
```

前端配置：

```
NEXT_PUBLIC_API_BASE_URL=http://127.0.0.1:8000
```

后端配置：

```
TWINLOOP_CORS_ORIGINS=http://127.0.0.1:3000
TWINLOOP_FRONTEND_URL=http://127.0.0.1:3000
ZHIHU_OAUTH_REDIRECT_URI=http://127.0.0.1:8000/api/v1/sources/zhihu/callback
```

当前 CORS 已经支持：

```
Origin: http://127.0.0.1:3000
Access-Control-Allow-Credentials: true
```

三、知乎 OAuth 登录流程

1. 前端检查登录状态

前端打开页面时调用：

```
GET /api/v1/me
```

完整地址：

```
http://127.0.0.1:8000/api/v1/me
```

请求带上：

```
Cookie: twinloop_session=...
```

如果用户未登录，后端返回：

```
401 Unauthorized
```

这是正常现象，前端会显示登录页面。

如果已经登录，返回类似：

```
{
  "data": {
    "user_id": "用户ID",
    "avatar_id": null,
```

```
"avatar_status": "not_created",
"zhihu_connected": true,
"zhihu": {
  "fullname": "知乎昵称",
  "avatar_url": "头像地址",
  "headline": "个人签名"
}
```

2. 点击知乎登录

前端调用：

```
POST /api/v1/sources/zhihu/connect
```

请求体：

```
{
  "scopes": [
    "profile",
    "answers"
  ]
}
```

后端生成：

- `state`
- 知乎授权地址
- OAuth 回调地址

返回类似：

```
{
  "data": {
    "authorization_url": "https://openapi.zhihu.com/authorize?...",
    "state": "随机状态值"
  }
}
```

前端随后执行：

```
window.location.href = authorization_url;
```

浏览器跳转到知乎授权页面。

3. 知乎授权完成后回调

知乎回调：

```
GET /api/v1/sources/zhihu/callback
```

回调地址：

```
http://127.0.0.1:8000/api/v1/sources/zhihu/callback
```

回调参数可能是：

```
authorization_code=xxx  
state=xxx
```

后端执行：

1. 校验 `state`
2. 使用授权码换取知乎 Access Token
3. 获取知乎用户资料
4. 根据知乎 UID 创建或更新本地用户
5. 创建服务端会话
6. 写入 HttpOnly Cookie：

```
twinloop_session
```

7. 302 跳转到：

```
http://127.0.0.1:3000
```

浏览器后续访问 API 时会自动携带 Cookie。

四、知乎资料接口

正式知乎资料接口位于：

```
/api/v1/zhihu
```

这些接口要求用户已经登录，并且后端会从服务端会话读取知乎 Token。

1. 读取知乎创作内容

```
GET /api/v1/zhihu/contents
```

参数：

```
type=all  
limit=20  
offset=0  
sort=ts  
order=desc
```

支持的内容类型：

```
all  
answer  
article  
zvideo  
pin  
question
```

2. 读取关注列表

```
GET /api/v1/zhihu/followees
```

参数：

```
limit=20  
offset=0
```

3. 读取收藏夹

```
GET /api/v1/zhihu/favlists
```

参数：

```
limit=20
```

4. 读取指定收藏夹内容

```
GET /api/v1/zhihu/favlists/{url_token}/contents
```

5. 读取近期收藏

```
GET /api/v1/zhihu/collections
```

如果知乎授权过期，后端返回：

```
401  
ZHIHU_TOKEN_EXPIRED
```

前端应引导用户重新授权。

五、数字分身初始化流程

初始化接口位于：

```
/v1/twin/initializations
```

1. 创建初始化会话

```
POST /v1/twin/initializations
```

请求体：

```
{  
  "import_job_id": "job_xxx"  
}
```

后端状态通常按以下方向推进：

```
importing  
personality_pending  
domain_pending  
opinion_questions_pending  
generating_profile  
review  
completed  
failed
```

同一个用户只能有一个初始化会话。

2. 查询初始化状态

```
GET /v1/twin/initializations/{session_id}
```

返回：

- 当前状态
- 当前步骤
- 已提交的数据
- 生成的画像结果
- 创建时间和更新时间

3. 提交初始化步骤

```
POST /v1/twin/initializations/{session_id}/{step}
```

请求体：

```
{  
  "data": {}  
}
```

step 可以是：

```
personality
domains
opinion-answers
social-answers
```

例如提交性格结果：

```
{
  "data": {
    "answers": {
      "q1": 4,
      "q2": 2
    },
    "notes": "补充说明"
  }
}
```

4. 完成初始化

```
POST /v1/twin/initializations/{session_id}/complete
```

请求体：

```
{
  "identity": {
    "display_name": "林同学",
    "summary": "关注技术和社会问题",
    "occupation": "学生",
    "location": "北京"
  }
}
```

后端会：

1. 读取性格测试结果
2. 读取兴趣和专长
3. 读取观点题结果
4. 读取社交情景题结果
5. 创建唯一数字分身
6. 创建第一个 active 画像版本
7. 保存原始输入为来源资料
8. 写入兴趣、专长、观点、行为记忆
9. 返回头像 ID 和版本 ID

核心数据库包括：

```
users
user_avatars
avatar_versions
avatar_identity
avatar_personality
avatar_styles
avatar_memories
source_documents
memory_evidence
avatar_policies
avatar_memory_rules
```

六、性格测评接口

1. 获取题目

```
GET /v1/personality/questions
```

2. 提交测评

```
POST /v1/personality/assessments
```

请求体：

```
{
  "answers": {
    "q1": 4,
    "q2": 3
  },
  "notes": "可选说明",
  "request_key": "前端幂等键"
}
```

3. 跳过测评

```
POST /v1/personality/skip
```

4. 查询最近一次测评

```
GET /v1/personality/assessments/latest
```

七、兴趣和专长领域接口

接口前缀：

```
/v1/domains
```

1. 查询领域目录

```
GET /v1/domains
```

支持查询参数：

```
q
parent_id
level
```

2. 查询当前用户选择

```
GET /v1/domains/selections/me
```

3. 保存兴趣和专长

```
PUT /v1/domains/selections/me
```

请求体：

```
{
  "expected_revision": 0,
  "interests": [
    {
      "domain_id": "人工智能",
      "level": "interested"
    }
  ]
}
```

```
    }  
  ],  
  "expertise": [  
    {  
      "domain_id": "软件工程",  
      "level": "熟悉"  
    }  
  ]  
}
```

4. 查询领域详情

```
GET /v1/domains/{domain_id}
```

5. 查询领域相关知乎问题

```
POST /v1/domains/{domain_id}/research
```

该接口用于初始化阶段生成观点选择题的素材。

八、旧版导入和画像原型接口

当前项目还保留了一组早期原型接口：

```
POST /v1/consents  
POST /v1/import-jobs  
GET /v1/import-jobs/{tid}  
GET /v1/profile  
PATCH /v1/profile  
POST /v1/versions  
GET /v1/versions/{vid}
```

其中：

```
POST /v1/import-jobs
```

当前主要是原型任务流程，返回：

```
queued  
fetching
```

```
storing_raw  
extracting  
review  
completed
```

前端个人中心目前仍会调用：

```
/v1/import-jobs
```

因此这个接口目前和正式知乎读取接口并存。后续如果要完全统一，可以把前端初始化入口改为正式的 `/v1/twin/initializations` 流程。

九、场景接口

1. 获取场景列表

```
GET /v1/twin/scenes
```

返回：

```
{
  "items": [
    {
      "id": "cafe",
      "name": "咖啡馆"
    },
    {
      "id": "library",
      "name": "图书馆"
    },
    {
      "id": "bar",
      "name": "小酒馆"
    },
    {
      "id": "theater",
      "name": "戏剧院"
    },
    {
      "id": "lecture",
      "name": "讲座"
    }
  ]
}
```

2. 获取场景空闲 Agent

```
GET /v1/twin/scenes/{scene_id}/avatars
```

后端会：

- 查询场景内 **idle** 的 Agent
- 排除当前用户自己的 Agent
- 返回展示名称和基础摘要
- 不返回敏感画像

对应数据库：

```
agent_scene_presence
```

状态包括：

```
idle
busy
offline
```

十、三种匹配模式

1. 指定模式

请求：

```
{
  "scene_id": "library",
  "match_mode": "manual",
  "target_avatar_id": "avatar_b"
}
```

后端校验：

- B 是否存在
- B 是否属于当前场景
- B 是否空闲
- B 是否不是当前用户自己的 Agent

2. 随机模式

请求：

```
{
  "scene_id": "library",
  "match_mode": "random"
}
```

后端从当前场景空闲池随机选择 B。

3. LLM 匹配模式

请求：

```
{
  "scene_id": "library",
  "match_mode": "llm"
}
```

后端读取候选的基础画像摘要，调用回答模型选择最适合的 B。

如果 LLM 配置不可用，当前实现会降级为随机匹配。

4. 匹配接口

```
POST /v1/twin/matches
```

返回：

```
{
  "scene_id": "library",
  "match_mode": "random",
  "initiator": {
    "user_id": "user_a",
    "avatar_id": "avatar_a",
    "display_name": "我的看山"
  },
  "invited": {
    "avatar_id": "avatar_b",
    "display_name": "小陈",
    "status": "idle"
  }
}
```

当前用户的 Agent 始终是：

A: 发起方

匹配到的其他 Agent 始终是：

B: 被邀请方

十一、双 Agent 对话流程

1. 创建对话

```
POST /v1/twin/dialogues
```

请求体：

```
{
  "scene_id": "library",
  "match_mode": "manual",
  "target_avatar_id": "avatar_b",
  "initial_question": null,
  "max_rounds": 10
}
```

前端不能提交发起方 avatar，后端从当前登录用户会话中读取 A。

后端执行：

1. 获取当前用户 A
2. 根据模式选择 B
3. 创建 `dialogue_run_id`
4. 创建 `conversation_id`
5. 创建 `chat_no`
6. 将 A 和 B 从 `idle` 改为 `busy`
7. 创建后台 `asyncio Task`
8. 立即返回运行 ID

返回：

```
{
  "run_id": "run_xxx",
  "avatar_a_id": "avatar_a",
  "avatar_b_id": "avatar_b",
  "scene_id": "library",
  "status": "queued"
}
```

2. 话题门控

对话正式开始前：

1. 读取 A 的高置信度观点
2. 读取 B 的高置信度观点
3. 使用 LLM 找双方观点的相似主题
4. 有共同主题时优先使用共同主题
5. 没有共同主题时，从 A 的观点或场景中随机选题
6. B 判断自己是否感兴趣

兴趣阈值：

```
DIALOGUE_INTEREST_THRESHOLD=0.55
```

如果 B 兴趣分数低于阈值：

- B 生成自然拒绝
- 写入一条拒绝消息
- 对话状态变为 `completed`
- 结束原因写入：

```
topic_rejected
```

不会继续普通聊天。

3. 正常交替对话

默认最多 10 轮：

```
A 开场  
B 回答  
A 回应或追问  
B 回答  
...
```

最多产生约 20 条 Agent 消息。

每条消息执行：

1. 当前 Agent 重新组装自己的画像上下文
2. 显式检索自己的记忆和聊天历史
3. 调用自己的 LLM
4. 如果产生行为记忆工具调用，写入行为记忆
5. 工具调用完成后重新准备上下文
6. 生成最终文本
7. 写入 PostgreSQL
8. 尝试写入 Chroma
9. 发布 SSE 事件

A 和 B 的上下文相互隔离：

```
A 只能读取 A 的画像和记忆  
B 只能读取 B 的画像和记忆
```

4. 提前结束

如果模型输出包含类似：

```
再见
先聊到这里
下次再聊
结束对话
```

系统会提前完成对话。

用户也可以调用：

```
POST /v1/twin/dialogues/{run_id}/cancel
```

十二、实时对话 SSE 流程

1. 建立 SSE 连接

```
GET /v1/twin/dialogues/{run_id}/events
```

前端通过：

```
new EventSource(
  `${API_BASE}/v1/twin/dialogues/${runId}/events`
)
```

接收事件。

2. 事件类型

```
run_started
topic_selected
message
round_progress
evaluation
status
error
```

事件示例：

```
{
  "event_id": "event_000003",
```

```
"event": "message",
"run_id": "run_xxx",
"round": 2,
"max_rounds": 10,
"speaker": "B",
"status": "running",
"payload": {
  "content": "我觉得这个问题很有意思。"
}
```

3. 断线续传

客户端可以发送：

```
Last-Event-ID: event_000003
```

或者：

```
/v1/twin/dialogues/{run_id}/events?last_event_id=event_000003
```

服务端会返回之后的事件，前端按 `event_id` 去重。

十三、对话状态查询

```
GET /v1/twin/dialogues/{run_id}
```

返回：

- 当前状态
- 对话结果
- 聊天号
- 已产生事件

状态可能包括：

```
queued
running
evaluating
completed
evaluation_failed
```

```
cancelled
failed
```

十四、评判和匹配度通知

正常对话结束后：

1. 状态改为 `evaluating`
2. 调用独立评判模型
3. 对完整聊天记录评分
4. 保存到 `agent_dialogue_evaluations`
5. 状态变为 `completed` 或 `evaluation_failed`
6. 如果分数超过 `GREAT_SCORE`，调用空推送接口

默认：

```
GREAT_SCORE=0.8
```

当前推送实现：

```
NoopMatchPushGateway
```

只记录调用参数，不发送真实通知。

话题拒绝的零轮对话不会触发高质量匹配通知。

十五、聊天组和聊天记录

一次 `run_agent_dialogue()` 调用对应一个聊天组。

相关表：

```
agent_chat_groups
chat_conversations
chat_participants
chat_messages
```

1. 查询聊天组数量

```
GET /v1/twin/chat-groups/count
```

可选：

```
status=completed
```

只统计当前用户作为以下任一角色的聊天：

```
initiator_user_id  
invited_user_id
```

2. 查询聊天组列表

```
GET /v1/twin/chat-groups?page=1&page_size=20&status=completed
```

只返回：

```
processing_status=ready  
visible_at IS NOT NULL
```

返回内容包括：

- 聊天号
- 发起方 Agent
- 被邀请方 Agent
- 状态
- 开始时间
- 结束时间
- 消息数量
- 是否未读
- 未读数量

3. 查询完整聊天记录

```
GET /v1/twin/chat-groups/{chat_no}/messages
```

后端校验当前用户必须是：

```
initiator_user_id = 当前用户  
OR invited_user_id = 当前用户
```

消息按照：

```
sequence_no ASC
```

排序。

软删除消息不会返回。

4. 标记已读

```
POST /v1/twin/chat-groups/{chat_no}/read
```

写入：

```
agent_chat_group_reads
```

保存：

- 首次查看时间
- 最近查看时间

十六、次日流程

对话完成、取消或失败后，前端调用：

```
POST /v1/twin/dialogues/{run_id}/advance-day
```

后端会：

1. 检查对话是否已经处于终态
2. 当前用户天数加一
3. 清空当天场景
4. 保存上一轮运行 ID
5. 将日状态设置为：

```
selecting
```

前端重新回到地图/场景选择页面。

对应数据库：

```
agent_world_days
```

十七、行为记忆和混合检索流程

每次 Agent 回答前都会执行：

```
固定画像 PostgreSQL 查询
+
关键词检索
+
Chroma 向量检索
+
PostgreSQL 权限校验
+
聊天历史检索
```

检索内容包括：

```
experience
fact
opinion
behavior
expertise
interest
source_document
chat_history
```

Chroma 只保存：

- ID
- 相似度
- avatar_id
- conversation_id
- 模型和向量维度等元数据

完整正文始终从 PostgreSQL 读取。

如果 Agent 使用行为记忆工具：

```
save_behavior_memory
```

会写入：

```
avatar_memories
```

状态默认是：

```
unconfirmed
```

十八、数据库脚本执行顺序

建议执行：

```
psql -U postgres -d zhihu -f E:\ZH\Zhihu\backend\db\postgresql_zhihu_schema.sql
psql -U postgres -d zhihu -f E:\ZH\Zhihu\backend\db\postgresql_chat_schema.sql
psql -U postgres -d zhihu -f E:\ZH\Zhihu\backend\db\postgresql_agent_dialogue.sql
psql -U postgres -d zhihu -f
E:\ZH\Zhihu\backend\db\postgresql_agent_chat_groups.sql
psql -U postgres -d zhihu -f
E:\ZH\Zhihu\backend\db\postgresql_twin_initialization.sql
psql -U postgres -d zhihu -f E:\ZH\Zhihu\backend\db\postgresql_agent_presence.sql
psql -U postgres -d zhihu -f
E:\ZH\Zhihu\backend\db\postgresql_agent_chat_group_reads.sql
```

其中：

```
postgresql_zhihu_schema.sql
```

创建画像基础表。

```
postgresql_chat_schema.sql
```

创建聊天会话和消息表。

```
postgresql_agent_dialogue.sql
```

创建双 Agent 运行和评判表。

```
postgresql_agent_chat_groups.sql
```

创建面向用户查询的聊天组表。

```
postgresql_twin_initialization.sql
```

创建初始化、心灵感应反馈、性格调整表。

```
postgresql_agent_presence.sql
```

创建场景占用和用户日状态表。

```
postgresql_agent_chat_group_reads.sql
```

增加聊天组可见状态和用户已读状态。

十九、启动流程

启动后端

```
cd E:\ZH\Zhihu  
.\backend\run_server.ps1
```

后端监听：

```
127.0.0.1:8000
```

启动前端

```
cd E:\ZH\Zhihu\frontend  
npm.cmd install  
npm.cmd run dev
```

前端监听：

```
127.0.0.1:3000
```

访问地址

当前本机直连模式访问：

```
http://127.0.0.1:3000
```

不使用 Nginx 时，所有 API 请求由前端直接发往：

```
http://127.0.0.1:8000
```

二十、当前项目需要特别注意的地方

当前项目已经包含主要闭环，但代码中还保留部分历史原型逻辑：

1. 正式登录接口是：

```
/api/v1/me
```

2. 旧版原型身份接口是：

```
/v1/me
```

3. 前端个人中心目前仍调用旧版：

```
/v1/import-jobs
```

4. 正式知乎资料读取使用：

```
/api/v1/zhihu/*
```

5. 初始化、场景、对话和聊天记录使用：

```
/v1/twin/*
```

6. 未登录时：

```
GET /api/v1/me → 401
```

是正常响应，不是后端故障。

7. 如果看到：

```
OPTIONS /api/v1/sources/zhihu/connect → 400
```

通常是前端 Origin、CORS 配置或后端服务未重启导致的。当前代码已经把 Origin 末尾斜杠自动清理，修改 `.env` 后仍需重启后端。